Rapport Labo 4 - Computer Architectuur

Victor Deforce en Cían Olieslagers

March 9, 2025

1 Probleemstelling

1.1 Opgave

In deze labosessie onderzoeken we hoe verschillende geheugenstructuren en toegangspatronen de uitvoeringstijd beïnvloeden. Dit gebeurt aan de hand van twee oefeningen:

- **Coalesced Memory Access**: We analyseren hoe coalesced en niet-coalesced geheugenaccesses de prestaties beïnvloeden bij het verwerken van een afbeelding.
- **Matrixvermenigvuldiging met verschillende geheugentypes**: We vergelijken de uitvoeringstijden van een matrixvermenigvuldigingskernel die uitsluitend global memory gebruikt met varianten die shared en constant memory benutten.

2 Implementatie

2.1 Coalesced memory access

Om het effect van coalesced en niet-coalesced geheugenaccesses te analyseren, hebben we twee CUDA-kernels geïmplementeerd die de rode kleurcomponent van een afbeelding inverteren.

- **Niet-coalesced geheugenaccess** (`InvertImageNtCoalesced`)

  De eerste functie verwerkt de afbeelding door elke thread een sprong van `C` bytes te laten maken bij het lezen en schrijven van pixelwaarden. Hierdoor worden de geheugenadressen die opeenvolgende threads benaderen niet netjes uitgelijnd in het geheugen, wat leidt tot inefficiënte geheugenaanvragen.

- **Coalesced geheugenaccess** (`InvertImageCoalesced`)

  In deze versie zorgt de implementatie ervoor dat elke thread direct een opeenvolgende geheugenlocatie aanspreekt. Dit betekent dat de threads in een warp hun geheugenverzoeken kunnen combineren tot een enkele efficiënte DRAM-toegang, wat de toegangstijd aanzienlijk verlaagt.

Voor beide functies wordt een afbeelding gegenereerd en in het GPU-geheugen geladen. Vervolgens wordt de uitvoer terug naar de CPU gekopieerd om de resultaten te analyseren. Na de verwerking wordt het toegewezen geheugen correct vrijgemaakt om geheugenlekken te voorkomen.

2.2 Matrixvermenigvuldiging met verschillende geheugentypes

Voor de implementatie van matrixvermenigvuldiging hebben we drie CUDA-kernels ontwikkeld, waarbij verschillende geheugenstrategieën worden gebruikt.

- **Matrixvermenigvuldiging met global memory** (`matrixMulGlobal`)

  In deze functie krijgt elke thread de verantwoordelijkheid voor het berekenen van één element in de uitvoermatrix `C`. Elke thread bepaalt de rij- en kolomindex op basis van de thread- en block-ID en haalt vervolgens de benodigde waarden op uit de inputmatrices `A` en `B` uit global memory. Een lus wordt gebruikt om het inwendig product van de rij van `A` en de kolom van `B` te berekenen. De berekende waarde wordt uiteindelijk opgeslagen in de corresponderende positie in matrix `C`.

- **Matrixvermenigvuldiging met shared memory** (`matrixMulShared`)

  In deze implementatie wordt shared memory gebruikt om herhaaldelijke toegang tot global memory te verminderen. De invoermatrices worden in kleinere tegels opgesplitst. Elke thread laadt een deel van matrix `A` en matrix `B` in een gedeelde geheugenbuffer. Een synchronisatiepunt (`__syncthreads()`) zorgt ervoor dat alle threads hun deel van de gegevens hebben geladen voordat de berekening plaatsvindt. Vervolgens wordt de matrixvermenigvuldiging in delen uitgevoerd, waarbij de berekende deelresultaten worden opgeteld. Uiteindelijk wordt de definitieve waarde in matrix `C` geschreven.

- **Matrixvermenigvuldiging met constant memory** (`matrixMulConstant`)

  In deze kernel wordt de tweede matrix `B` in constant memory opgeslagen. Tijdens de uitvoering haalt elke thread de corresponderende waarden uit matrix `A` uit global memory en de waarden uit matrix `B` uit constant memory. De berekening verloopt op dezelfde manier als in de global memory-implementatie, waarbij een lus wordt gebruikt om het inwendig product van de rij van `A` en de kolom van `B` te berekenen. De eindwaarde wordt in matrix `C` geplaatst.

Voor alle implementaties worden de inputmatrices gegenereerd op de CPU en vervolgens overgezet naar het GPU-geheugen. De uitvoering van de kernels gebeurt binnen een rooster van blokken en threads, afhankelijk van de matrixgrootte. Na de berekening wordt de uitvoermatrix teruggekopieerd naar de CPU. Tenslotte worden alle toegewezen geheugenbuffers correct vrijgegeven om geheugenlekken te voorkomen.

3 Observatie

3.1 Memory coalescence

We zien hier dat het accessen van geheugenvelden die elkaar opvolgen sneller gebeurd dan het accessen van niet opeenvolgende adressen. Dit valt te verklaren doordat de GPU reeksen adressen tegelijk uit geheugen kan laden, waardoor er voor meerdere threads tegelijk geheugen kan worden uitgelezen. Als de accesses niet sequentieel zijn wordt hier geen gebruik van gemaakt. We zien echter ook dat het effect van memory coalescence pas onderscheidbaar wordt bij voldoende grote arrays. Uit de grafiek kunnen we aflezen dat dit vanaf 10^8 elementen is. Dit valt te verklaren doordat er pas bij die grootteordes alle cores gebruikt worden en dus de sequentiële DRAM accesses van één core de bottleneck worden voor de andere.

3.2 Memory Types

Hier zien we weinig verschil tussen de verschillende types. Dit komt omdat we de test hebben uitgevoerd met een matrix van 32x32, waardoor er in verhouding met andere overhead de reads van memory slechts een kleine rol speelden. De verwachte resultaten bij grotere matrices zijn dat de shared memory opstelling het snelst is doordat de waarden in on-device geheugen zitten met een kleine overhead tot gevolg. De constant data verwachten we op tweede plaats aangezien deze off-device is maar meer geoptimaliseerd is dan global memory. Het gebruik van beide is gelimiteerd door de grootte van de cache.

4 Conclusie

Bij het uitvoeren van memory intensive kernels is het aangewezen om memory coalescence toe te passen om reeksen geheugen locaties in blokken te kunnen inladen. Zo kunnen er meer DRAM accesses gebeuren per seconde en vormt minder snel een bottle-neck. Voor lichtere toepassingen moet men hier minder op letten. Daarnaast kan de memory access korter in tijd door de data eerst over te zetten naar shared of constant geheugen. Hier is het ook pas aan te raden eens er voldoende accesses moeten gebeuren, anders weegt de overhead van het overzetten niet op tegen de korter durende accesses. Verder zijn deze veel kleiner dan global memory.

5 Appendix - code

5.1 Deel 1: Coalesced Memory Access

5.1.1 Niet Coalesced Memory acces:

```
#define C 3
 __global__ void InvertImageNtCoalaseerd(uint8_t* image_out, uint8_t* image_in, int size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < size/C) {  // Prevent out-of-bounds access
        image_out[idx*C] = 255 - image_in[idx*C];
    }

}

int sizes[][2] = { {8192, 16384}, {2048, 4096}, {4096, 8192}, {4096,8192} };

    for (int test = 0; test < 4; test++) {
        int M = sizes[test][0];
        int N = sizes[test][1];
        int size = M * N * C;

        uint8_t* h_image_in = (uint8_t*)malloc(size);
        uint8_t* h_image_out = (uint8_t*)malloc(size);
        for (int i = 0; i < size; i++) {
            h_image_in[i] = rand() % 256;
        }

        uint8_t* d_image_in, * d_image_out;
        cudaMalloc((void**)&d_image_in, size);
        cudaMalloc((void**)&d_image_out, size);

        cudaMemcpy(d_image_in, h_image_in, size, cudaMemcpyHostToDevice);

        int threads_per_block = 256;
        int num_pixels = size / C;
        int num_blocks = (num_pixels + threads_per_block - 1) / threads_per_block;

        auto start2 = std::chrono::high_resolution_clock::now();
        InvertImageNtCoalaseerd <<<num_blocks, threads_per_block >>> (d_image_out, d_image_in, size);
        cudaDeviceSynchronize();
        auto end2 = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double> elapsed2 = end2 - start2;

        cudaMemcpy(h_image_out, d_image_out, size, cudaMemcpyDeviceToHost);

        std::cout << "M: " << M << ", N: " << N << std::endl;
        std::cout << "Coalesced: " << elapsed1.count() << " seconds" << std::endl;
        std::cout << "Non-Coalesced: " << elapsed2.count() << " seconds" << std::endl;

        cudaFree(d_image_in);
        cudaFree(d_image_out);
        free(h_image_in);
        free(h_image_out);
    }
```

5.1.2 Coalesced Memory acces:

```
#define C 3
__global__ void InvertImageCoalaseerd(uint8_t* image_out, uint8_t* image_in, int size)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < size / C) { // Per pixel
        image_out[idx] = 255 - image_in[idx]; // Alleen rood aanpassen
    }


}
int sizes[][2] = { {8192, 16384}, {2048, 4096}, {4096, 8192}, {4096,8192} };

    for (int test = 0; test < 4; test++) {
        int M = sizes[test][0];
        int N = sizes[test][1];
        int size = M * N * C;

        uint8_t* h_image_in = (uint8_t*)malloc(size);
        uint8_t* h_image_out = (uint8_t*)malloc(size);
        for (int i = 0; i < size; i++) {
            h_image_in[i] = rand() % 256;
        }

        uint8_t* d_image_in, * d_image_out;
        cudaMalloc((void**)&d_image_in, size);
        cudaMalloc((void**)&d_image_out, size);

        cudaMemcpy(d_image_in, h_image_in, size, cudaMemcpyHostToDevice);

        int threads_per_block = 256;
        int num_pixels = size / C;
        int num_blocks = (num_pixels + threads_per_block - 1) / threads_per_block;

        auto start1 = std::chrono::high_resolution_clock::now();
        InvertImageCoalaseerd <<<num_blocks, threads_per_block >>> (d_image_out, d_image_in, size);
        cudaDeviceSynchronize();
        auto end1 = std::chrono::high_resolution_clock::now();
        std::chrono::duration<double> elapsed1 = end1 - start1;

        cudaMemcpy(h_image_out, d_image_out, size, cudaMemcpyDeviceToHost);

        std::cout << "M: " << M << ", N: " << N << std::endl;
        std::cout << "Coalesced: " << elapsed1.count() << " seconds" << std::endl;
        std::cout << "Non-Coalesced: " << elapsed2.count() << " seconds" << std::endl;

        cudaFree(d_image_in);
        cudaFree(d_image_out);
        free(h_image_in);
        free(h_image_out);
    }
```

5.2 Deel 2: Matrixvermenigvuldiging met verschillende geheugentypes

5.2.1 Global Memory

```
#define TILE_SIZE 3
float** allocateMatrix(int rows, int cols) {
    float** matrix = new float* [rows];
    for (int i = 0; i < rows; i++) {
        matrix[i] = new float[cols];
    }
    return matrix;
}

// Functie om een 2D-matrix om te zetten naar een 1D-array
void flattenMatrix(float** matrix, float* array, int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            array[i * cols + j] = matrix[i][j];
        }
    }
}
__global__ void matrixMulGlobal(float* A, float* B, float* C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < width && col < width) {
        float sum = 0.0f;
        for (int i = 0; i < width; i++) {
            sum += A[row * width + i] * B[i * width + col];
        }
        C[row * width + col] = sum;
    }
}
 float h_A[N][N] = {
        {-1, -1,  4},
        { 0,  3, -3},
        { 2, -1, -2}
    };

    float h_B[N][N] = {
        {3, 2, 3},
        {2, 2, 1},
        {2, 1, 1}
    };

    float h_C[N][N] = { 0 }; // Output matrix
    float* h_A_flat = (float*)malloc(N * N * sizeof(float));
    float* h_B_flat = (float*)malloc(N * N * sizeof(float));
    float* h_C_flat = (float*)malloc(N * N * sizeof(float));

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            h_A_flat[i * N + j] = h_A[i][j];
            h_B_flat[i * N + j] = h_B[i][j];
        }
    }
```

```
float* d_A, * d_B, * d_C;
size_t size = N * N * sizeof(float);
cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);

cudaMemcpy(d_A, h_A_flat, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B_flat, size, cudaMemcpyHostToDevice);
cudaMemcpyToSymbol(B_const, h_B_flat, size); // Copy B to constant memory

dim3 threadsPerBlock(N, N);
dim3 numBlocks(1, 1); // Since it's a 3x3 matrix

matrixMulGlobal << <numBlocks, threadsPerBlock >> > (d_A, d_B, d_C, N);
cudaDeviceSynchronize();
cudaMemcpy(h_C_flat, d_C, size, cudaMemcpyDeviceToHost);
std::cout << "Global Memory Result:\n";
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        std::cout << h_C_flat[i * N + j] << " ";
    }
    std::cout << std::endl;
}
cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
free(h_A_flat);
free(h_B_flat);
free(h_C_flat);

return 0;
```

5.2.2 Constant Memory

```
#define N 3 // Matrix size (N x N)
__constant__ float B_const[N * N];

__global__ void matrixMulConstant(float* A, float* C, int width) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < width && col < width) {
        float sum = 0.0f;
        for (int i = 0; i < width; i++) {
            sum += A[row * width + i] * B_const[i * width + col];
        }
        C[row * width + col] = sum;
    }
}
cudaMemcpyToSymbol ( B_const, h_B_flat , N*N * sizeof ( float ) );
matrixMulConstant << <numBlocks, threadsPerBlock >> > (d_A, d_C, N);
cudaDeviceSynchronize();
cudaMemcpy(h_C_flat, d_C, size, cudaMemcpyDeviceToHost);
std::cout << "Constant Memory Result:\n";
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        std::cout << h_C_flat[i * N + j] << " ";
    }
    std::cout << std::endl;
}
```

5.2.3 Shared Memory

```
_global__ void matrixMulShared(float* A, float* B, float* C, int width) {
    __shared__ float A_tile[TILE_SIZE][TILE_SIZE];
    __shared__ float B_tile[TILE_SIZE][TILE_SIZE];
    int row = threadIdx.y;
    int col = threadIdx.x;
    A_tile[row][col] = A[row * width + col];
    B_tile[row][col] = B[row * width + col];
    __syncthreads();
    float sum = 0.0f;
    for (int i = 0; i < width; i++) {
        sum += A_tile[row][i] * B_tile[i][col];
    }
    __syncthreads();
    C[row * width + col] = sum;
}
matrixMulShared << <numBlocks, threadsPerBlock >> > (d_A, d_B, d_C, N);
cudaDeviceSynchronize();
cudaMemcpy(h_C_flat, d_C, size, cudaMemcpyDeviceToHost);
std::cout << "Shared Memory Result:\n";
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        std::cout << h_C_flat[i * N + j] << " ";
    }
    std::cout << std::endl;
}
```

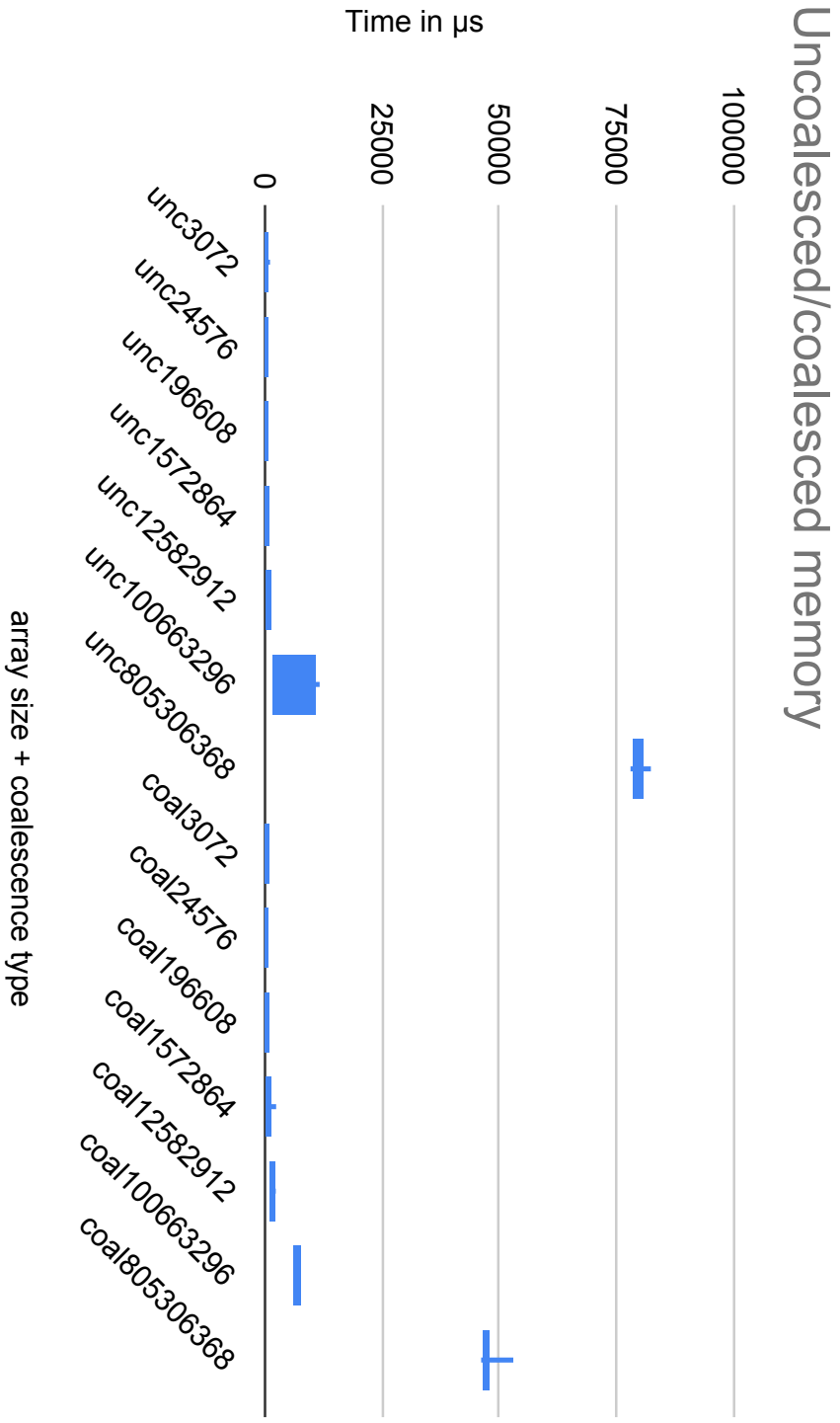

Uncoalesced/coalesced memory
Time in µs
100000
75000
50000
25000
0
unc3072
unc24576
unc196608
unc1572864
unc12582912
unc100663296
unc805306368
coal3072
coal24576
coal196608
coal1572864
coal12582912
coal100663296
coal805306368
array size + coalescence type

Figure 1:

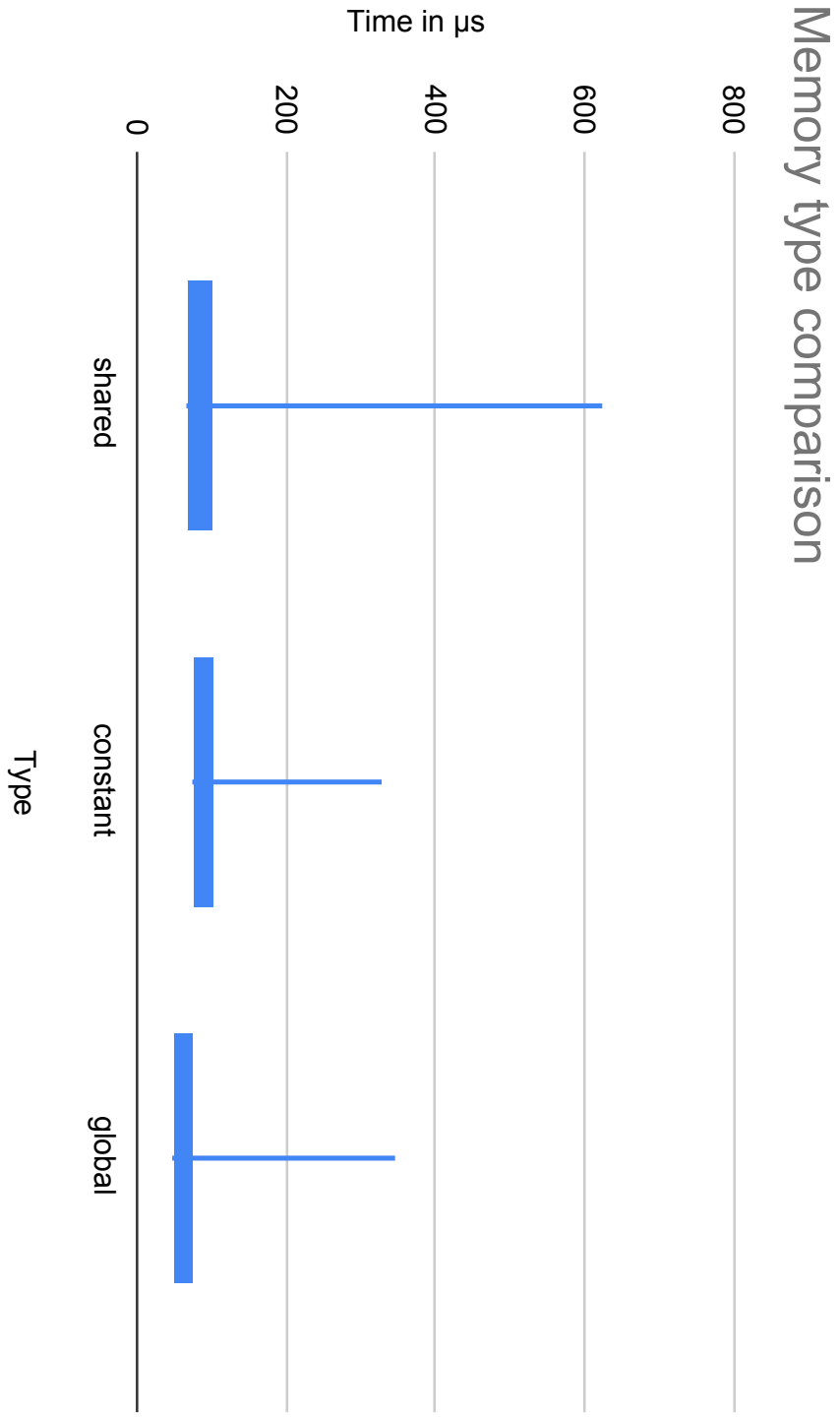

Memory type comparison
Time in µs
0
200
400
600
800
Type
shared
constant
global

Figure 2:

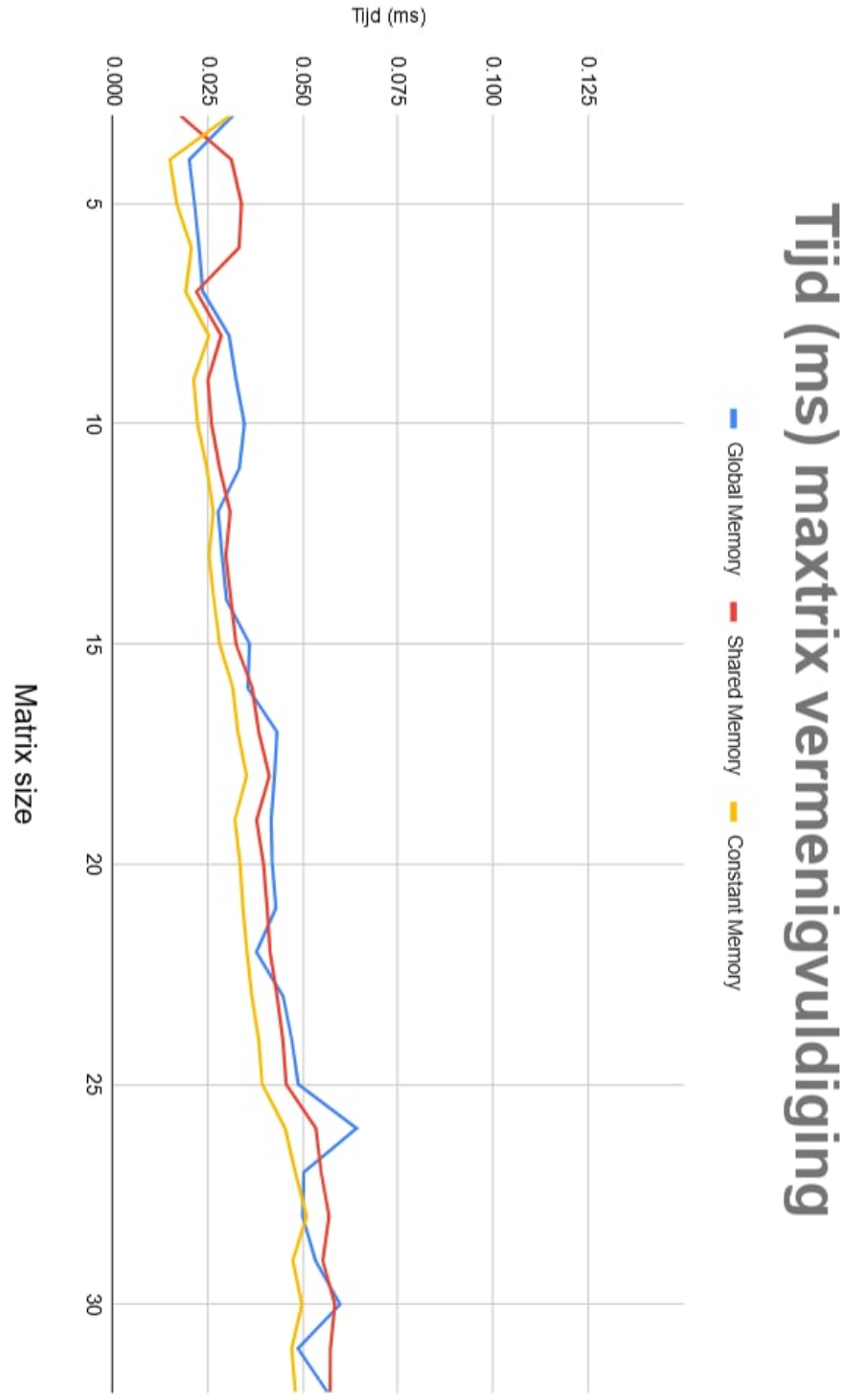

Tijd (ms) maxtrix vermenigvuldiging
Global Memory
Shared Memory
Constant Memory
Tijd (ms)
0.000
0.025
0.050
0.075
0.100
0.125
5
10
15
20
25
30
Matrix size

Figure 3: